SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr.T.P. Anithaashri

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Experiments **Duration: 3hrs**

1. **Design and develop** a **Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

(10 Marks)

2. **Design and implement** an **Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

(10 Marks)

3. **Develop** a **Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

(10 Marks)

4. **Design and develop** a **Proximal Policy Optimization (PPO)** model for controlling **Smart** HVAC **Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

(10 Marks)

5. **Implement** a **Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

(10 Marks)

6. **Design and develop** a **Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

(10 Marks)

7. **Develop** a **Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

(10 Marks)

8. **Design and implement** an **A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

(10 Marks)

9. **Develop** a **Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.

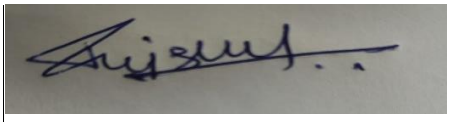
(10 Marks)

10. **Design, implement, and evaluate** a **Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.

(10 Marks)

Code of Conduct:

I B. Rajesh (192472165) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Intelligent Traffic Signal Control

Question: Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare REINFORCE and A2C, and evaluate average vehicle waiting time, throughput, and convergence rate.

About the Question:

This question asks us to use Reinforcement Learning to control traffic signals automatically. Instead of a person deciding when the traffic light should turn green or red, an RL agent learns the decision from traffic conditions. The main aim is to reduce the time vehicles wait at signals and allow more vehicles to pass through the road.

RL Components:

Agent: Traffic signal controller that decides the signal action.

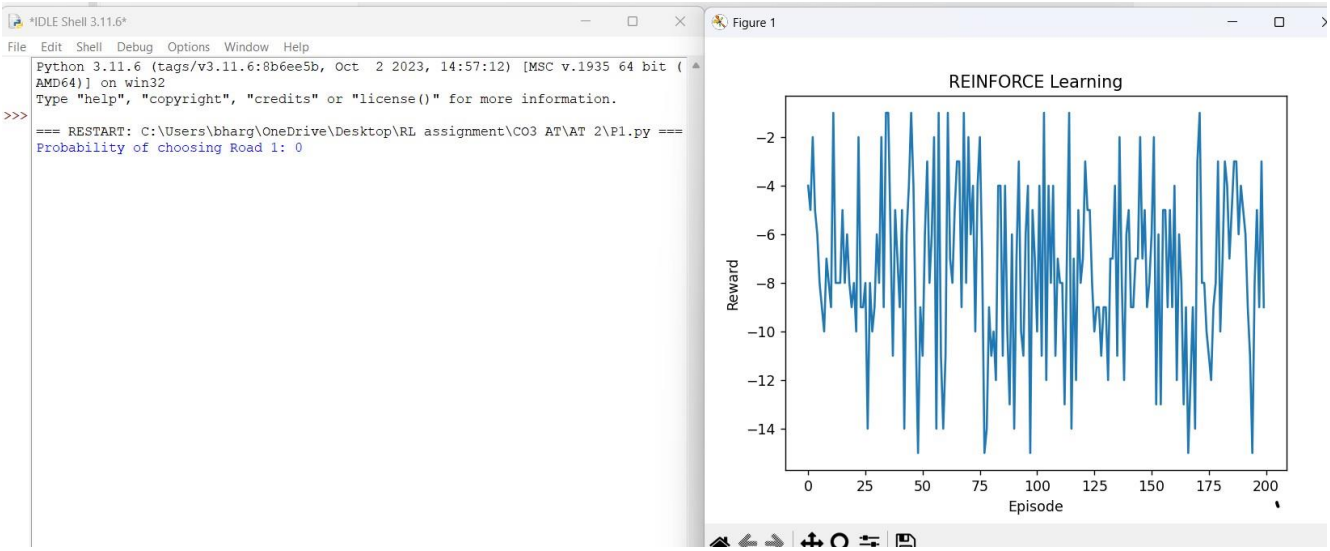
Environment: The road intersection and traffic around it.

State: Current traffic condition, such as number of vehicles or queue length.

Action: Change or maintain the traffic signal.

Reward: Positive reward for reducing waiting time and improving traffic flow; penalty for congestion.

Goal: Reduce average waiting time, increase throughput, and achieve stable learning.



Autonomous Warehouse Robot

Question: Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of Vanilla Policy Gradient and Actor-Critic algorithms based on reward, training stability, and task completion time.

About the Question:

2.

This question asks us to make a warehouse robot learn how to collect and deliver packages efficiently. The robot observes the warehouse situation and chooses movements. Actor-Critic methods use an Actor to choose actions and a Critic to evaluate those actions. The experiment compares this approach with Vanilla Policy Gradient.

RL Components:

Agent: Autonomous warehouse robot.

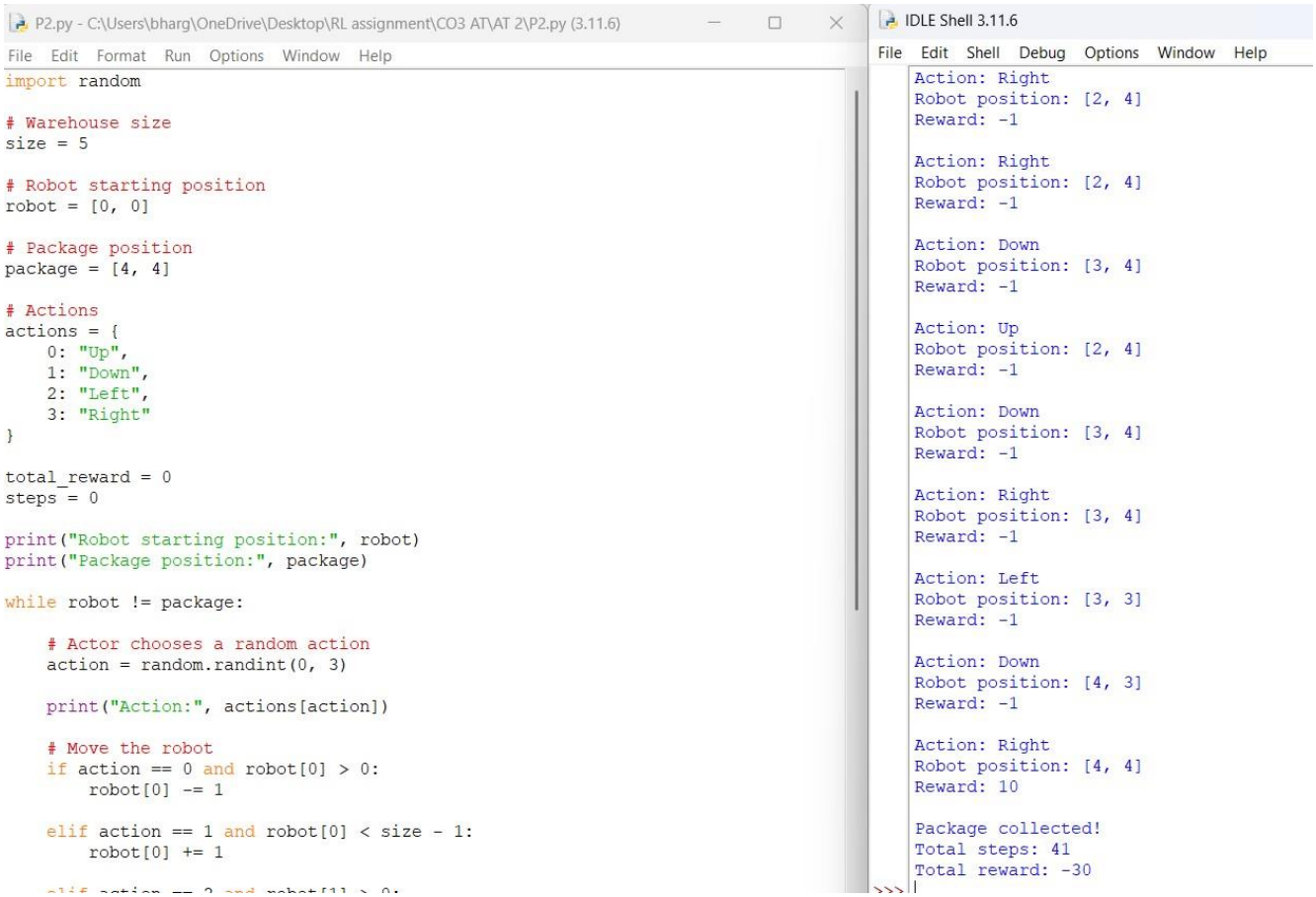
Environment: Warehouse with shelves, paths, packages, and delivery locations.

State: Robot position, package location, destination, and nearby obstacles.

Action: Move in different directions, collect a package, or deliver it.

Reward: Positive reward for successful delivery and efficient movement; penalty for delays or collisions.

Goal: Increase reward, improve training stability, and reduce task completion time.



DDPG Portfolio Optimization

Question: Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

About the Question:

3.

This question applies DDPG to stock portfolio management. DDPG is useful because the action can be continuous, such as deciding what percentage of money to invest in an asset. The system learns investment decisions from market information while trying to increase returns and control risk.

RL Components:

Agent: Portfolio trading decision system.

Environment: Stock market or a simulated market.

State: Market prices, returns, portfolio value, and other selected market information.

Action: Continuous portfolio allocation, such as the percentage invested in an asset.

Reward: Profit or return, possibly adjusted by risk or transaction cost.

Goal: Increase cumulative return while controlling risk and achieving stable convergence.

4.

Smart HVAC Energy Management

Question: Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

About the Question:

This question asks us to automatically control a building's HVAC system using PPO. The controller should not simply minimize electricity use; it must also keep the building comfortable for occupants. PPO is compared with REINFORCE to see which learns better for this task.

RL Components:

Agent: Smart HVAC controller.

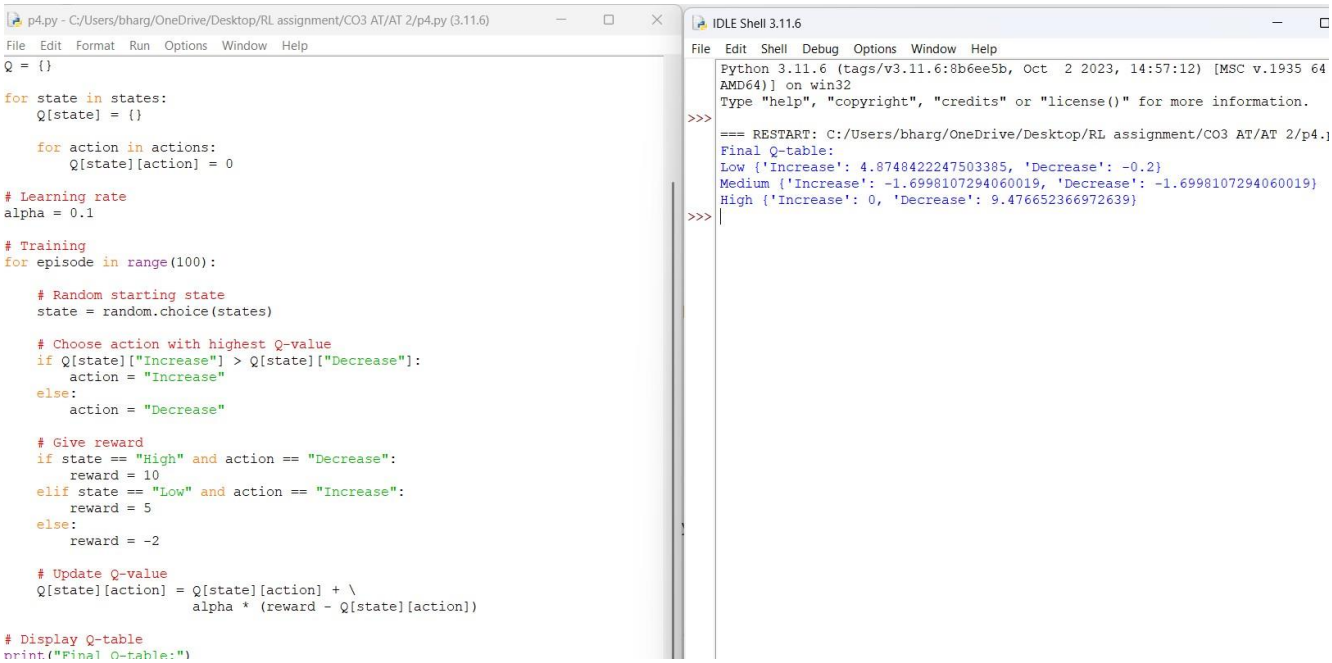
Environment: Smart building and its indoor conditions.

State: Indoor temperature, occupancy, and other relevant building conditions.

Action: Adjust heating, cooling, ventilation, or related HVAC settings.

Reward: Reward for maintaining comfort with low energy consumption; penalty for discomfort or excessive energy use.

Goal: Minimize energy consumption while maintaining occupant comfort.



Industrial Robotic Arm with TRPO

Question: Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling an Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

5.

About the Question:

This question asks us to use TRPO to control an industrial robotic arm. The robot should learn accurate movements for manufacturing tasks. TRPO is a policy-optimization method designed to make controlled policy updates, which can help maintain stable learning.

RL Components:

Agent: Industrial robotic arm controller.

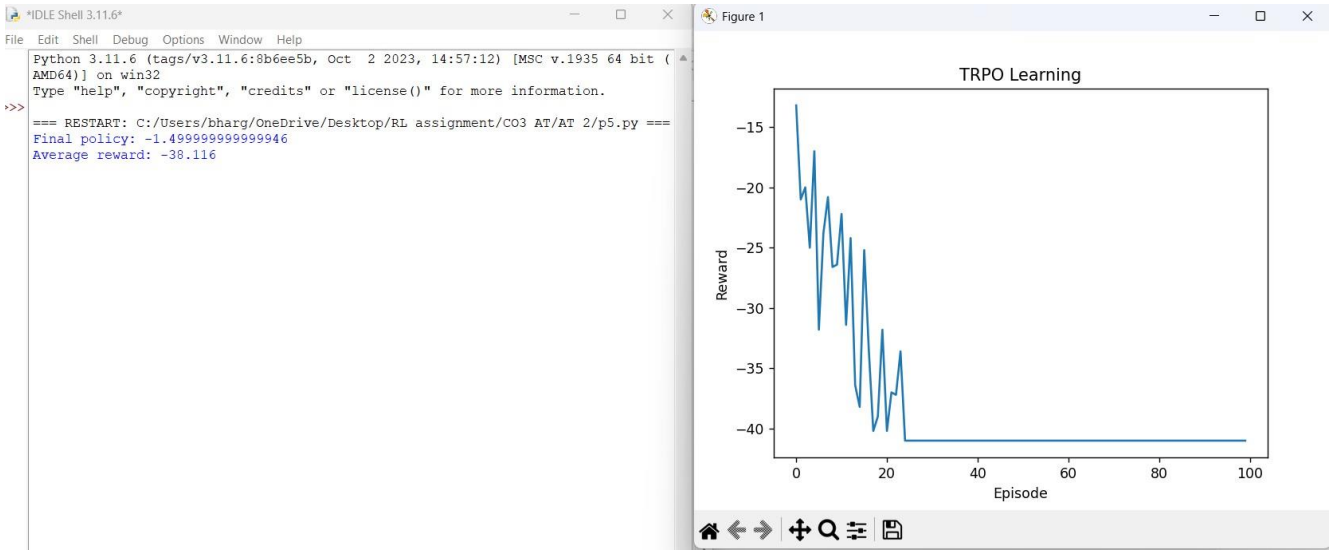
Environment: Automated manufacturing environment.

State: Robot position, joint angles, target position, and task condition.

Action: Robot movement or joint-control commands.

Reward: Positive reward for accurate target movement and successful production; penalty for errors or unsafe movement.

Goal: Improve policy stability, precision, convergence speed, and production efficiency.



Adaptive Patient Treatment Planning

Question: Design and develop a Policy-Based Reinforcement Learning system for Healthcare adaptive patient treatment planning. Compare A2C and PPO in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

About the Question:

This question asks for a policy-based RL system using simulated patient data. The system learns treatment decisions from simulated patient conditions and rewards. A2C and PPO are then compared. This is an educational simulation, not a real clinical decision system.

RL Components:

6.

Agent: Simulated treatment decision system.

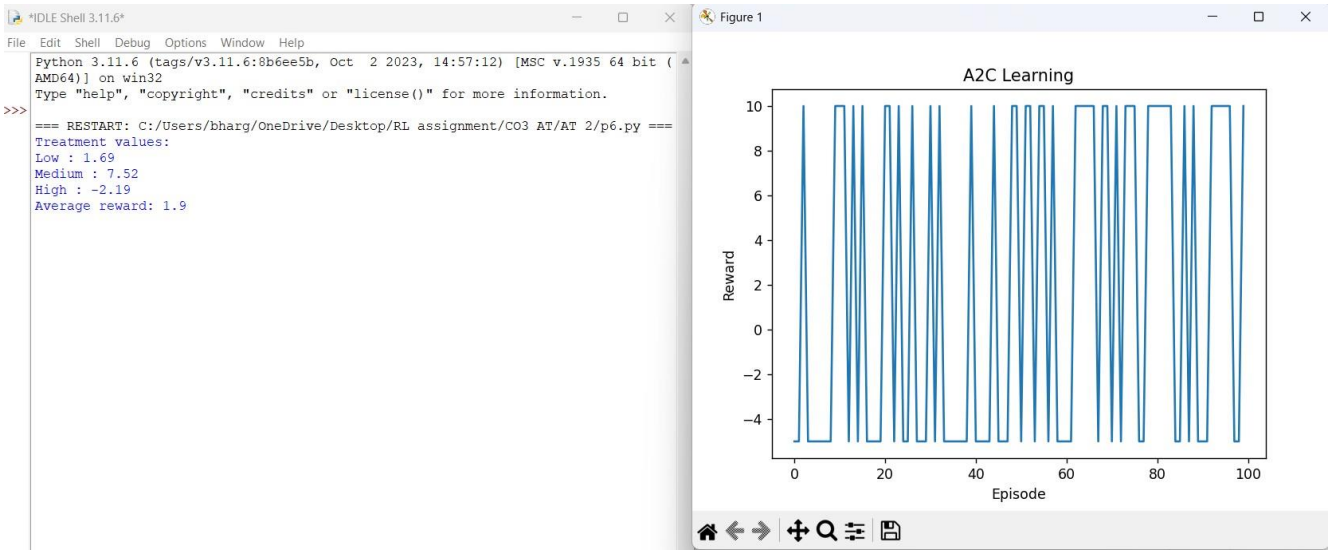
Environment: Simulated healthcare/patient environment.

State: Simulated patient condition and relevant treatment-related variables.

Action: Select a treatment option in the simulation.

Reward: Reward for improved simulated treatment outcomes; penalty for poor simulated outcomes.

Goal: Improve simulated treatment effectiveness, cumulative reward, and learning stability.



Autonomous Drone Navigation

Question: Develop a Deep Reinforcement Learning model using DDPG or PPO for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

About the Question:

This question asks us to make a drone learn how to navigate through an environment that can change. The drone must reach its destination while avoiding obstacles and using energy efficiently. DDPG or PPO can be selected as the learning algorithm.

RL Components:

Agent: Autonomous drone.

Environment: Dynamic environment containing a route, target, and changing obstacles.

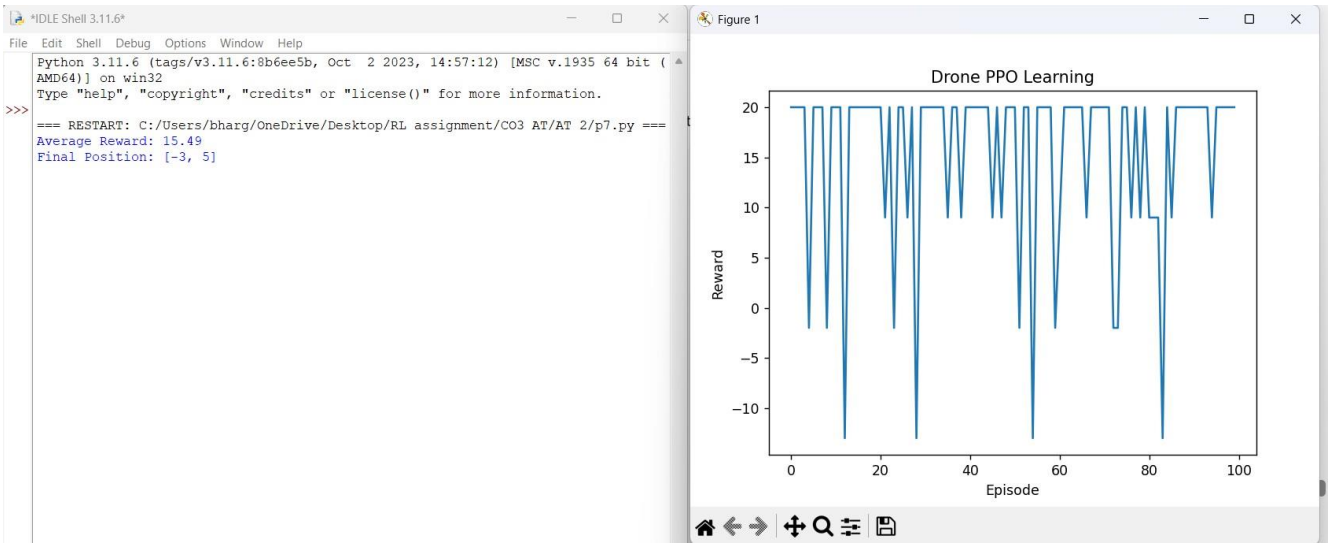
State: Drone position, target position, obstacle information, and movement-related information.

Action: Drone movement or control commands.

Reward: Positive reward for reaching the target and safe movement; penalty for collisions, unnecessary movement, or high energy use.

7.

Goal: Improve navigation accuracy, obstacle avoidance, energy efficiency, and policy convergence.



Dynamic Cloud Resource Allocation

Question: Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

About the Question:

This question asks us to use A3C to decide how cloud resources should be allocated dynamically. The system observes resource usage and learns how to allocate resources according to demand. A3C uses multiple learning workers, while Vanilla Policy Gradient provides the comparison.

RL Components:

Agent: Cloud resource allocation controller.

Environment: Cloud computing system and its workloads.

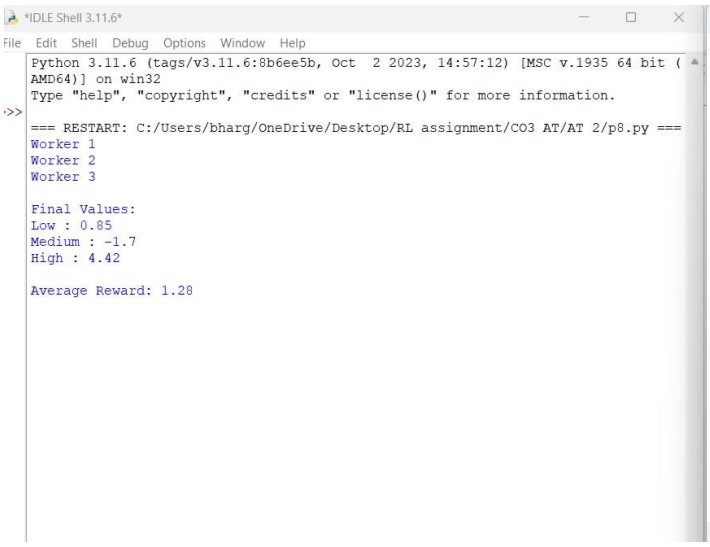
State: CPU utilization, workload level, and available resources.

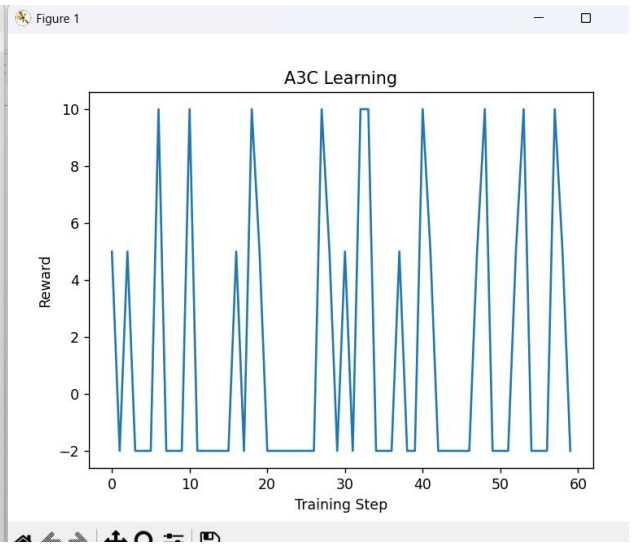
Action: Increase, decrease, or adjust allocated resources.

Reward: Reward for efficient allocation and good response time; penalty for waste, overload, or poor response.

Goal: Reduce response time, maintain suitable CPU utilization, and improve resource efficiency.

8.





Predictive Maintenance for Industrial Machines

Question: Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of REINFORCE and PPO.

About the Question:

This question asks us to predict when industrial machines need maintenance using a policy-gradient RL system. Instead of waiting for a machine to fail, the system learns when maintenance should be performed. The reward function should balance maintenance cost against the cost of machine downtime.

RL Components:

Agent: Predictive maintenance decision system.

Environment: Industrial machine and its operating condition.

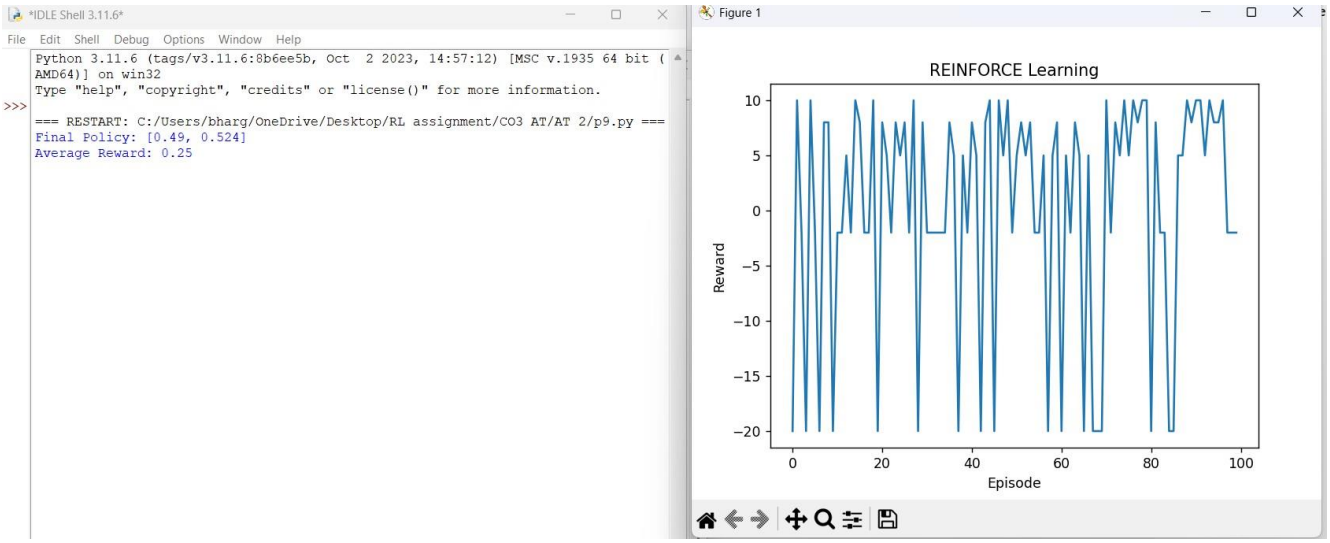
State: Machine condition and sensor-related information.

Action: Perform maintenance or do not perform maintenance, according to the designed action space.

Reward: Reward for preventing failures at reasonable cost; penalty for unnecessary maintenance and machine downtime.

Goal: Minimize maintenance cost and machine downtime.

9.



Autonomous Lane-Keeping

Question: Design, implement, and evaluate a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using TRPO or PPO. Compare the selected algorithm with A2C based on lane deviation, safety, reward convergence, and training efficiency.

About the Question:

This question asks us to teach an autonomous vehicle to stay safely inside its lane. The controller observes the vehicle's position relative to the lane and chooses steering actions. TRPO or PPO can be selected, and its performance must be compared with A2C.

RL Components:

Agent: Autonomous vehicle lane-keeping controller.

Environment: Road and lane environment.

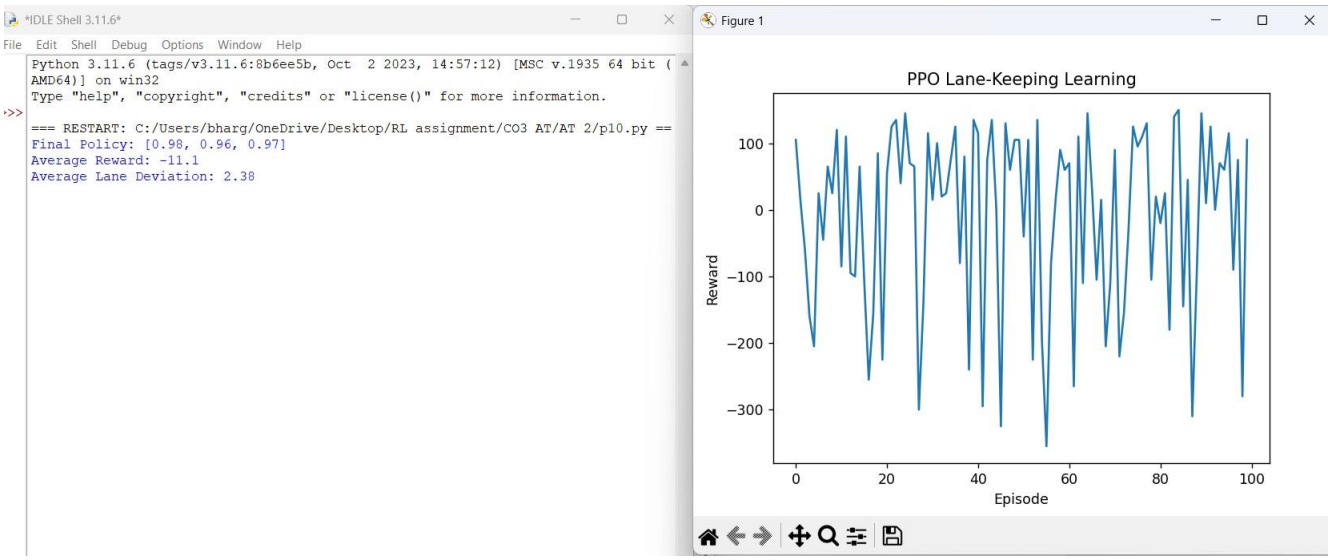
State: Vehicle position or deviation from the lane center and other relevant driving information.

Action: Steer left, keep straight, or steer right, depending on the designed action space.

Reward: Positive reward for staying safely near the lane center; penalty for large lane deviation or unsafe behavior.

Goal: Reduce lane deviation, improve safety, achieve reward convergence, and improve training efficiency.

10.


IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:/Users/bharg/OneDrive/Desktop/RL assignment/CO3 AT/AT 2/p10.py ==
Final Policy: [0.98, 0.96, 0.97]
Average Reward: -11.1
Average Lane Deviation: 2.38
Figure 1
PPO Lane-Keeping Learning
Reward
Episode
100
0
−100
−200
−300
0
20
40
60
80
100